DevSync

رواد مصر الرقمية

GRADUATION PROJECT

Literature Review.

DevSync: Secure AI Project Collaborator
Feedback, improvements and grading criteria

TEAM OF 5

Donia Alhosin
Bishay Talaat
Yousef Mohy
Emad Hany
Saga Maher

REPORT DURATION

9 Oct 2026 - 16 Oct 2026

1 week

About this version

The research behind DevSync is unchanged. This version ties the evidence plan, improvements and grading evidence to the **feature owners**, and aligns the task-management roadmap with the if-time task board. Lecturer fields (Parts 2 and 4) are left blank on purpose.

Required component	Where to find it
Literature review	Part 1
Feedback and Evaluation (lecturer)	Part 2
Suggested Improvements	Part 3
Final Grading Criteria	Part 4
Future enhancements: task management	Part 5
References	End

Part 1: Literature review

This review asks why developers struggle to find fitting open-source projects, what existing tools offer, and which recommendation, AI and engineering methods suit a small Flutter and Firebase app. Please check each reference (authors, year, venue, DOI) against the original before submission; platform descriptions reflect our reading in Oct 2026.

Why newcomers struggle. Steinmacher et al. (2015) collected the barriers newcomers face in open source: finding a project and a first task, understanding the codebase, getting community help, and having the right technical background. Finding a suitable project is the problem DevSync targets. Maintainers face the reverse: contributors with matching skills are hard to find and assess.

Existing tools

Tool	What it offers	Limit	How DevSync differs
GitHub Explore, topics, good first issue	Trending repositories; issues marked beginner friendly	Filters by label or topic, not by the person; no fit score; no request flow	Scores each project against the user's GitHub activity and explains it
Up For Grabs	Curated list of beginner-friendly projects	Static; depends on owners tagging tasks; no personal ranking	Personal ranked feed that improves with like/skip
CodeTriage	Subscribe to issues of chosen repositories	User must already know which repositories	Helps discover repositories first
Good-first-issue search engines	Filter issues by language and label	Issue-level only; no collaboration conversation	Project fit plus collaboration requests with real-time alerts

Recommendation approaches. Ricci et al. (2015) group recommenders as content-based, collaborative filtering and hybrid. A new app has a cold-start problem, so DevSync starts content-based (GitHub data and declared skills) and stores like/skip signals so a hybrid approach is possible later.

LLMs for matching. Models can compare a structured profile with a project and return a score and short reason without custom training. Few-shot prompting fixes the output format (Brown et al., 2020). Step-by-step reasoning can help on complex tasks (Wei et al., 2022), but DevSync asks only for a short reason to keep cost and latency low. Model-as-judge grading scales but has known biases (Zheng et al., 2023), so it is combined with a fixed evaluation set and real feedback.

LLM provider recommendation

Recommendation. Use **Google Gemini Flash-Lite** (currently Gemini 3.1 Flash-Lite, the cheapest current model) for the matching score and reason, called only from a Cloud Function. Keep the model name in configuration so it can be swapped. Use a hosted open-source model (Llama, Qwen or gpt-oss through a provider such as Groq or OpenRouter) as the backup option, and do not self-host.

Option	Fit for DevSync	Cost	Main risk
Gemini 3.1 Flash-Lite (recommended)	Fast and cheap for short scoring with a JSON answer; same Google/Firebase ecosystem as the backend; free tier for development	Free tier for testing; paid $0.25 per 1M input and $1.50 per 1M output tokens (Standard); Batch about half	Free-tier content is used to improve Google products, so use the paid tier once real pilot users' GitHub data is sent

Option	Fit for DevSync	Cost	Main risk
Gemini 3.5 Flash-Lite or 3.7 / 3.8 Flash	Better reasoning for tuning and comparison runs	Flash-Lite 3.5: $0.30 in, $2.50 out; Flash: $0.75 in, $3.75 out (to 31 Dec 2026, then double)	Higher cost and latency than the feed needs; use only if relevance on the evaluation set is clearly better
Hosted open-source (Llama, Qwen, gpt-oss via Groq or OpenRouter)	No vendor lock-in; very fast on some hosts; good as fallback and for the evaluation set	Per-token pricing and free tiers vary by host: confirm current prices before use	Rate limits, quality differences between models and hosts, and one more API key to protect
Self-hosted open-source	Full control over data	GPU server cost, not covered by the free Firebase quota	Setup time and cost are too high for the 30 Nov deadline; not recommended

Why this fits. The matching task is short: a profile and a few projects in, a score and one-sentence reason out. A small model is enough, and the evaluation set (target +15% relevance, 50% of top-5 rated relevant) decides whether a larger model is needed. Planning estimate: one feed run of about 2,500 input and 600 output tokens costs roughly $0.0015 on Gemini 3.1 Flash-Lite, so about 6,000 runs during the pilot cost around $9. Caching results per user and limiting calls per user keep it lower.

- Call the model only from a Cloud Function; the API key stays a function secret (or use a service account if Gemini is called through Google Cloud), never in the app or repository.
- Send only public GitHub data and declared skills (NFR-12); use the paid tier for pilot users.
- Run the evaluation set on Gemini 3.1 Flash-Lite and one open-source model in W3-W4 and keep the better one; the fallback ranking from skills stays in place whichever model is chosen.
- Owner: Emad (decide by end of W1 together with Donia for the Blaze plan and budget). Prices and model names change often, so check Google's pricing page before the final budget (last checked 9 Oct 2026).

How prompts are evaluated (Emad)

Step	What we do	Evidence
1. Fixed evaluation set	Sample profiles and projects with agreed relevance labels	Set in the repository
2. Score prompt versions	Run each version on the set	Relevance per version (target +15%, first to final)
3. Real feedback	Compare with pilot like/skip	50% or more of top-5 rated relevant
4. Keep the best	Keep and document the best version	AI documentation

Engineering and security. Firebase gives authentication, a document database and push messaging for a small team; Firestore rules decide who reads and writes each document. The OWASP MASVS gives the security checklist areas (storage, network, authentication, code quality), followed by Donia (rules, secrets) and Saga (checklist, obfuscated builds). Obfuscation makes reverse engineering harder but does not replace server-side checks. Scrum (Schwaber and Sutherland, 2020) suits one-week sprints.

Gaps and DevSync's response

Gap	Response	Owner
Discovery relies on labels and topics	Profile from GitHub activity and skills	Saga
No explanation of fit	Score and short reason on every match	Emad
No feedback loop	Like/skip feeds prompt evaluation	Yousef, Emad
Contact happens elsewhere	Requests with real-time alerts in the app	Yousef
Prompt quality rarely measured	Evaluation set, pilot feedback, KPIs	Emad
Security left to the end	Checklist, rules review, obfuscated build tested by 12 Nov	Donia, Saga

Finding a project is a real barrier, current tools do not rank for the individual, and content-based recommendation with LLM scoring is a practical start without history. DevSync adds a measured prompt-improvement loop.

Part 2: Feedback and evaluation

To be completed by the lecturer or supervisor. Ratings use 1-5 (1 = needs major work, 5 = excellent).

Field	Entry
Lecturer / supervisor	
Date of review	
Version reviewed	

Area assessed	What to look at	Rating (1-5)	Comments
Problem and idea	Clear problem, realistic scope, value for users		
Planning and management	Plan, roles, risks, KPIs, evidence of following the plan (feature papers, point loads)		
Requirements	Stakeholders, stories, functional and non-functional requirements		
Technical quality	Architecture, Firebase use, AI matching, code quality		
Security and release	Permissions, rules, obfuscation, release builds		
Testing	Coverage, bug list, prompt evaluation		
Documentation	README, technical documentation, report quality		
Presentation and demo	Clarity, working demo, answers to questions		

Overall comments, main strengths, main concerns and overall result: _____

Team evidence plan

Evidence	Collected from	When	Owner
Weekly progress summary, sprint report, points per person	Jira sprint reports, supervisor update	Every week	Donia
Demo of working features, one per owner	Sprint review call	End of each week	Each owner
Test and security report	CI results, bug list, security checklist	12 Nov, final 30 Nov	Saga (rules items by Donia)
Prompt improvement results	Evaluation set results by prompt version	12 Nov and 30 Nov	Emad
FCM delivery timing results	Delivery test script with timestamps	5 Nov and 12 Nov	Yousef
Pilot results (adoption, match quality, requests)	Firebase counts and like/skip	Late Nov	Donia, Emad
Git history and pull request review record	GitHub	Continuous	All

Lecturer suggestions and action tracker

Suggestion from lecturer	Action we will take	Owner	Due	Status

Part 3: Suggested improvements

"Now" items can still fit before 30 Nov; "Future" items follow the deadline. Effort: S = under 1 day, M = 2-4 days, L = more than 4 days. Owners follow the feature papers; owners of future items are proposed.

Area	Improvement	Effort	When	Owner
Security	Keep the LLM key in a Cloud Function secret, not in the app	S	Now	Emad (function), Donia (secrets scan)
AI	Compare Gemini Flash-Lite with a hosted open-source model on the evaluation set and keep the better one	S	Now	Emad
AI	Fallback ranking from skills when the LLM is slow or fails	S	Now	Emad
AI	Cache match results and limit calls per user	S	Now	Emad
Quality	Run the evaluation set automatically in CI	M	Future	Emad, with Yousef
AI	Embedding pre-filter, then LLM ranking of top candidates	M	Future	Emad
AI	Issue-level matching and skill-gap suggestions	M	Future	Emad

Area	Improvement	Effort	When	Owner
AI	Remote Config prompt versions with A/B tests	M	Future	Emad, with Yousef
Product	Maintainer search, in-app chat, endorsements	L	Future	Team
Product	Arabic localization with right-to-left layout	M	Future	Bishay
Product	Onboarding tour and shareable profile card	S	Future	Bishay
Trust	Report and block users, GitHub-verified profiles, Firebase App Check	M	Future	Donia (rules), Yousef (App Check)
Platform	Web platform on the same backend, then iOS	L	Future	Yousef
Business	Real Pro plan with in-app payments	L	Future	Yousef

Part 4: Final grading criteria

Marks are set by the course; fill them in from the course handbook.

Criterion	What is assessed	Evidence from DevSync	Marks
Documentation	Planning report, literature review, requirements, technical documentation, README, clean Git history	Three reports, README, architecture and data model docs (Donia), AI and prompt docs (Emad), each owner's feature documentation assembled by Saga	__ / __
Implementation	Working features; Flutter, Firebase Auth, FCM, animations, flavors, code quality	Working app with all must-have features, one owner per feature (Donia: auth, design system, rules; Bishay: projects, animations; Yousef: shell, requests and FCM, like/skip; Emad: AI matching and feed; Saga: profile, builds), Free and Pro flavors, pull request history	__ / __
Testing	Unit, widget and integration tests, manual testing, security review, prompt evaluation	Test and security report (Saga), CI results (Yousef), rules tests (Donia), evaluation set results (Emad), FCM delivery timings (Yousef)	__ / __
Presentation	Clear story, live demo, teamwork, answers	Final presentation and demo on 30 Nov, led by Bishay, with every member presenting their own features	__ / __
Total			__ / __

Workload evidence: Jira epics with points per person (core total 47: Donia 12 including the finished design system, Bishay 7, Emad 8, Yousef and Saga 10) and the pull request record.

Part 5: From the task board to a full task management system

The 30 Nov scope has only an optional shared board for two accepted collaborators: Donia owns the board (5 points), with task alerts by Yousef, a prompt dashboard by Emad, extra animations by Bishay and regression tests by Saga, all only after the 12 Nov core freeze. After that, the board can grow into a task system inside DevSync, because the literature shows contact between contributor and maintainer happens outside the tool.

Architecture. The app talks only to Firebase (Auth, Firestore, Cloud Messaging, Cloud Storage). Cloud Functions handle everything that needs a secret or an outside service (GitHub webhooks, LLM, Jira, digests). Secrets stay in function secrets, never in the app or repository.

Phased roadmap (planning estimates)

Phase	Timing	Deliverables	Done when
1: Foundation	Dec 2026	Workspaces per accepted collaboration, full task model, roles and permissions, task alerts (builds on Donia's board and Yousef's alerts)	Two collaborators can plan and finish a project from a workspace
2: Connected work	Jan-Feb 2027	Board, list and calendar views, comments, GitHub issue and pull request links	A merged pull request moves its task automatically
3: Planning and insight	Mar-Apr 2027	Sprints and backlog, dashboards, AI task assistant, Jira import	A team can run a sprint and read its report in the app
4: Reach	From May 2027	Live collaboration, calendar sync, offline mode, web version	The same workspace works on phone and desktop

Measure or risk	Target or response
Workspace adoption	At least 50% of accepted collaborations create a task within 24 hours
Task completion	At least 60% of tasks reach Done by their due date
GitHub sync accuracy	At least 95% of merged pull requests update the linked task
Scope growth	Ship in phases; each phase keeps the core stable
Notification overload	Per-user settings, daily digest, quiet hours
Permissions and data leaks	Rules unit tests for every role; audit log
Integration limits and cost	Cache GitHub data, batch calls, budget alerts, App Check and rate limits

References

- [1] Brown, T. B. et al. (2020). Language Models are Few-Shot Learners. NeurIPS 33.
- [2] OWASP Foundation. Mobile Application Security Verification Standard (MASVS). owasp.org.
- [3] Ricci, F., Rokach, L. and Shapira, B. (eds.) (2015). Recommender Systems Handbook, 2nd edition. Springer.
- [4] Schwaber, K. and Sutherland, J. (2020). The Scrum Guide. scrumguides.org.
- [5] Steinmacher, I., Graciotto Silva, M. A., Gerosa, M. A. and Redmiles, D. F. (2015). A systematic literature review on the barriers faced by newcomers to open source software projects. Information and Software Technology, 59.
- [6] Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 35.
- [7] Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023, Datasets and Benchmarks.
- [8] Official documentation: Flutter, Firebase (Authentication, Cloud Firestore, Cloud Messaging, Security Rules), Android developer guides. Accessed Oct 2026.
- [9] Platform pages: GitHub Explore and topics, Up For Grabs, CodeTriage. Accessed Oct 2026.